Libro di Testo — Informatica Classe 1

Corso di Informatica — classi 1, 2, 3, 4

Versione 0.9 · 28/10/2026

Corso a cura del prof. Nicola Regge

0. Come lavoriamo (leggere per prime)

1. Ogni lezione: **carta e penna sul banco** per appunti e schemi a mano.
2. Si lavora **un passo alla volta**, con un risultato concreto da vedere subito.
3. Consegne e materiali stanno su **Google Classroom**; i documenti si scrivono con **Google Documenti**.
4. **L'AI aiuta a capire, non a saltare il pensiero**: la prova che hai capito è **saperlo spiegare a voce** con parole tue.
5. Chi scrive meglio nella propria lingua può scrivere in **arabo/cinese** e poi tradurre in italiano con Google Traduttore.

1. Che cos'è l'informatica

1. L'informatica è la scienza che studia come **trattare le informazioni** con strumenti automatici: il computer riceve dati (input), li elabora e restituisce risultati (output).
2. Il computer da solo non "capisce": esegue **istruzioni** precise che gli diamo.
3. Impareremo a usare bene gli strumenti (computer, Google, Git) e, più avanti, a creare noi qualcosa (siti, giochi).

2. Il computer e le sue parti

1. **CPU (processore):** il "cervello", esegue tutti i calcoli.
2. **RAM (memoria):** memoria di lavoro, veloce ma **temporanea** (si svuota allo spegnimento).
3. **Hard Disk / SSD:** dove si salvano i file in modo **permanente** (restano anche da spento).
4. **Scheda madre:** la base che collega e fa comunicare tutti i pezzi.
5. **Scheda video (GPU):** disegna la grafica (immagini, video, giochi).
6. **Alimentatore:** porta la corrente a tutti i pezzi.
7. **Case:** la scatola che contiene e protegge tutto.

2.1 Come si assembla (dove va ogni pezzo)

1. La **scheda madre** si fissa alla parete di fondo del **case**.
2. La **CPU** va nello zoccolo (socket) al centro-alto della scheda madre; sopra il dissipatore con la ventola.
3. La **RAM** entra negli slot lunghi accanto alla CPU (fino al "clic").
4. La **scheda video** va nello slot PCIe, in basso sulla scheda madre.
5. L'**Hard Disk/SSD** nel vano del case, collegato con un cavo alla scheda madre.
6. L'**alimentatore** in basso nel case; dà corrente a tutti.

2.2 Scegliere i pezzi (compatibilità e budget)

1.
Con il sito `it.pcpartpicker.com/list/` (modalità **Builder**) si sceglie un pezzo per ogni categoria.

2.
Il sito **controlla la compatibilità** (avvisa in rosso se due pezzi non vanno insieme) e **somma il prezzo totale**.

3.
Obiettivo: montare un PC che funzioni **entro un budget** (es. 700 €).

2.3 Standard e compatibilità (form factor, PCIe, socket, RAM)

1.
Standard = misure e forme comuni, decise da tutti, perché i pezzi **combacino**. I quattro principali:

2.
Form factor (grandezza della scheda madre e del case): `ATX` (305×244 mm), `microATX` (244×244), `Mini-ITX` (170×170); `AT` è il vecchio standard. Più grande = più slot.

3.
Slot PCIe (schede aggiuntive): `x1, x4, x8, x16` (lunghezze crescenti); nel `x16` va la **scheda video**. Versioni `3.0/4.0/5.0` (velocità); `M.2` per gli SSD NVMe.

4.
Socket (dove si appoggia la CPU): Intel = `LGA` (pin nel socket, es. `LGA1700`); AMD = `PGA` (pin sulla CPU, es. `AM4`) o `LGA` (`AM5`). CPU e scheda madre si comprano **in coppia**.

5.
Processori per marca: Intel Core `i3/i5/i7/i9`; AMD Ryzen `3/5/7/9`.

6.
RAM (memorie): `DDR3/DDR4/DDR5`, con la **tacca in posizione diversa** (non si scambiano); `DIMM` (desktop) vs `SO-DIMM` (portatili).

7.
Cosa deve combaciare: casescheda madre (form factor), scheda madreCPU (socket), scheda madreRAM (generazione), scheda videoslot `PCIe x16`.

> **NOTA**
> È il “perché” dei controlli che fa PCPartPicker quando si monta il PC. Materiale: scheda `standard-hardware` (teoria trilingue con i disegni in scala).

2.4 Lo storage: i 5 livelli (dove si salvano i file)



I 5 livelli dello storage digitale

1.
Livello 1 — Tecnologia: l'**HDD** (meccanico) ha un piatto che gira e una testina che si muove; l'**SSD** (elettrico) usa **celle NAND Flash**, senza parti in movimento (più veloce e resistente).

2.
Livello 2 — Tipo e velocità: HDD meccanico ~160 MB/s; **SSD SATA** (2,5") ~560 MB/s; **SSD NVMe** molto più veloce (Gen3 ~3.500, Gen4 ~7.500, Gen5 ~12.000+ MB/s).

3.
Livello 3 — Logica di controllo (il "linguaggio"): AHCI (vecchio): seriale, una sola coda, va bene per l'HDD; **NVMe** (moderno): in parallelo, fino a 65.535 code, perfetto per i chip dell'SSD.

4.

Livello 4 — Connettore fisico: SATA (con cavo, robusto) oppure **M.2** (a incastro diretto sulla scheda madre, piccolo). Attenzione: **M.2 è solo una FORMA** del connettore (può essere SATA o NVMe).

5.

Livello 5 — Interfaccia (la "strada" sulla scheda madre): il **bus SATA** è lento e passa da un controller; il **bus PCIe** è velocissimo ed è collegato direttamente alla **CPU** (le "corsie" x4).

> **NOTA**
>
> Da ricordare, le tre cose diverse: **M.2** = la forma del connettore · **NVMe/AHCI** = il linguaggio · **PCIe/SATA** = la strada. Il disco più veloce oggi è un **SSD NVMe su M.2 collegato via PCIe**.

2.4.1 In parole semplici — la storia dello storage

1.

Una volta c'erano solo gli **HDD**: dischi che **girano**, con una **testina** che legge e scrive, un po' come la puntina di un giradischi. Funzionano, ma sono **lenti** e delicati, perché hanno **parti in movimento**.

2.

Poi sono arrivati gli **SSD**: **niente parti in movimento**, salvano tutto in **chip di memoria** (NAND Flash), come una chiavetta USB molto evoluta. Sono più **veloci**, silenziosi e resistenti agli urti.

3.

Ma non tutti gli SSD sono uguali. Un **SSD SATA** usa la **vecchia strada** dell'hard disk: è veloce, ma ha un limite (~560 MB/s). Un **SSD NVMe** usa una **strada nuova e larghissima**, il **PCIe** (la stessa della scheda video e della CPU), e va **molto** più veloce.

4.

Tre parole da **non confondere**:

5. la **FORMA** — `M.2`, il "bastoncino" che si incastra sulla scheda madre;
6. il **LINGUAGGIO** — `NVMe` o `AHCI`, cioè come il computer "parla" col disco;
7.

la **STRADA** — `PCIe` (autostrada) o `SATA` (strada normale), per dove passano i dati.

8.

In pratica: per far "volare" un computer, la prima cosa da fare è mettere un **SSD** (meglio se **NVMe**): è il salto di velocità che si sente di più, più ancora che cambiare processore.

> **NOTA**
>
> Immagine mentale: pensa a una **consegna di pacchi**. La **forma** è il tipo di furgone, il **linguaggio** è la lingua con cui parli al corriere, la **strada** è l'autostrada o la stradina di campagna. Per andare veloce servono tutte e tre giuste: furgone adatto (M.2), lingua moderna (NVMe), autostrada (PCIe).

3. Utenze e aree di lavoro

1.

Account e password: ognuno ha il proprio account della scuola; la password va tenuta al sicuro e robusta (lunga, con lettere, numeri e simboli).

2.

Le aree: cartella personale in rete + spazio su **Google Drive**; i file si organizzano in **cartelle e sottocartelle** (area logica).

3.

Chiudere bene il computer: quando hai finito, blocca o disconnetti con `Ctrl + Alt + Canc`; non lasciare il PC aperto e incustodito.

4. Google Suite e Classroom

1. Google Documenti: il programma per scrivere i testi (si salva da solo).

2.

Consegnare un compito su Classroom: aprire il corso “Lavori del corso” il compito “Aggiungi o crea” “Documenti Google” scrivere “Consegna”.

3.

Il voto: l’allievo vede il voto **solo dopo** che il docente clicca “Restituisci”.

5. GitHub e i repository

1.

GitHub è un sito dove i programmatori **salvano e condividono** il codice dei loro progetti e collaborano.

2.

Repository (repo): la “cartella-progetto” digitale che contiene tutti i file **e la storia di ogni modifica** fatta nel tempo.

3.

A cosa serve la storia delle modifiche: si può **tornare a una versione precedente** se qualcosa va storto, e si capisce chi ha cambiato cosa e quando.

4.

Git è il sistema che tiene traccia delle modifiche; **GitHub** è il sito che ospita i progetti online.

5.1 La storia del nostro gioco: versioni, rami, fork e merge (09/10/2026)

1. Versione (commit): ogni salvataggio pubblicato del gioco. Nel grafo è un pallino: v0.3, v1.1, v2.0.

2. Ramo (branch): una linea del grafo; il gioco cresce lungo il ramo, da sinistra a destra.

3. Fork (copia): la 1INF ha copiato il «Treno blindato» della 2INF (versione v0.11) e lo fa crescere per conto suo (v1.0, v1.1).

4. Pull Request (proposta di modifica): il Documento consegnato su Classroom («il mio cattivo si chiama Hidra…»): il prof lo legge, lo accetta e lo mette nel gioco.

5. Merge (unione): un ramo di lavoro rientra nel ramo principale. Il prof ha provato le «immagini della classe» su un ramo a parte, per non rompere il gioco; quando funzionavano le ha unite al ramo della 1INF.

6.

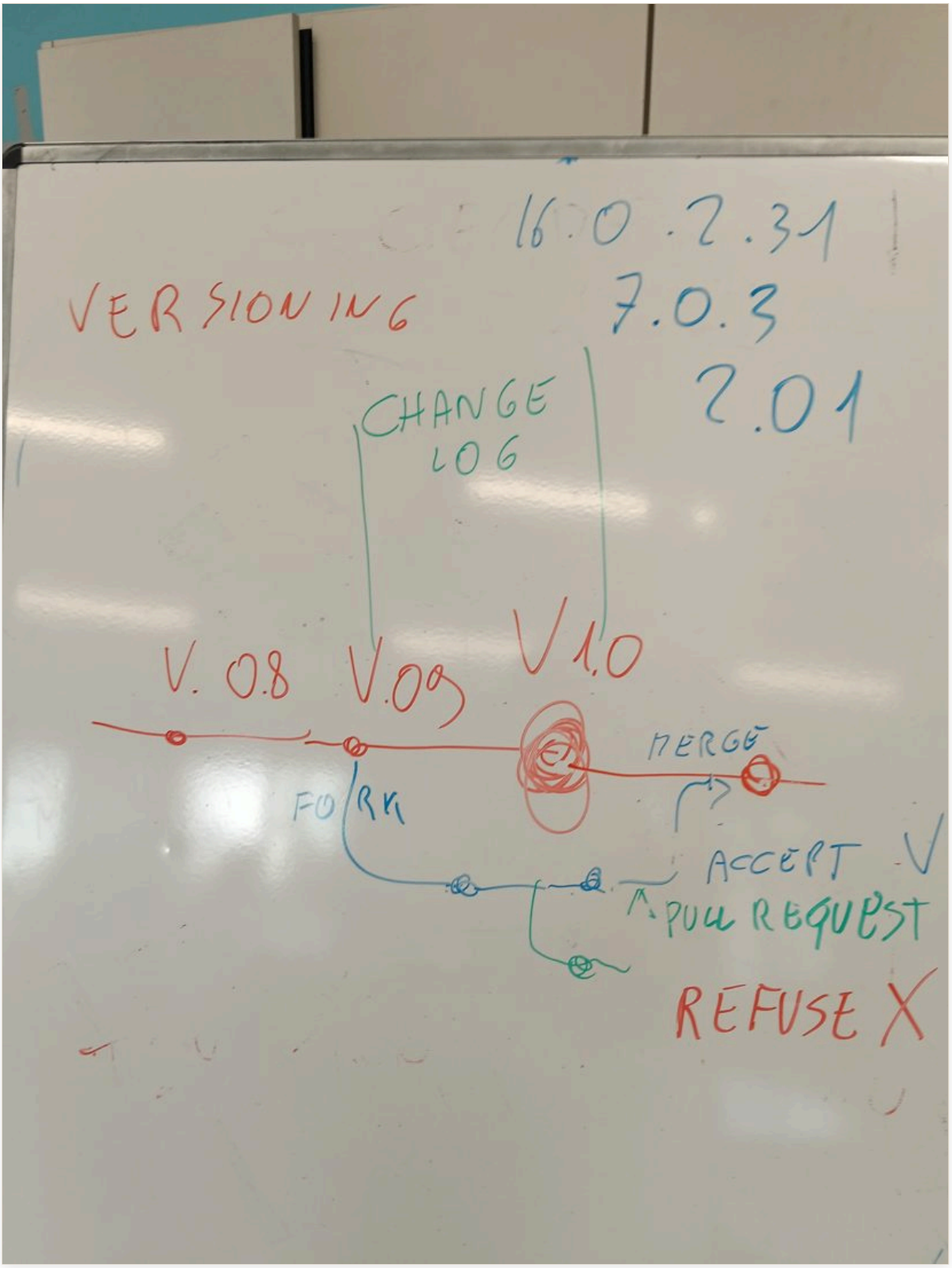
I numeri delle versioni: 6.1 v0.x = le prove, prima della versione stabile; 6.2 1.0 = la prima versione «vera» del ramo; 6.3 1.1, 1.2 = versioni **minor**: piccole aggiunte (un cattivo, un cielo); 6.4 2.0 = versione **major**: cambia una cosa grossa. Il nostro gioco è stato rifatto con **Godot** (motore per videogiochi): per questo è la v2.0 e non la v1.2.

7.

Ponte da Lazarus: Lazarus **reagisce** (aspetta un clic), Godot **pulsa** (`_process` gira da solo circa 60 volte al secondo).

8. Lezione completa (in italiano, arabo e cinese) e grafo: `classe-1/lezione-versioni-treno/` e la pagina `docs/lezione-versioni/`.

5.2 La lavagna del 09/10/2026: numeri delle versioni, fork, Pull Request



La lavagna del 09/10: versioning, changelog, fork, Pull Request accettata e rifiutata

1. I numeri delle versioni (in blu): 16.0.2.31 · 7.0.3 · 2.01. Il primo numero è la **major** (cambio grosso), il secondo la **minor** (aggiunte), il terzo la **patch** (piccole correzioni di errori), il quarto, se c'è, la **build** (numero di compilazione). Esempio: 16.0.2.31 = major 16, minor 0, patch 2, build 31.

2. Il ramo principale (in rosso): v0.8 v0.9 v1.0. Il cerchio grande è la v1.0, la prima versione stabile che si pubblica (**release**).

3. Il changelog (riquadro verde tra v0.9 e v1.0): la lista di tutto quello che è cambiato da una versione all'altra.

4. Il fork (in blu): dalla v0.9 qualcuno copia il progetto e lavora per conto suo.

5. La Pull Request: dal fork si propone la modifica al progetto principale; se va bene **accept** e **merge**, la modifica entra nel ramo rosso e nasce una versione nuova.

6. Refuse : una proposta che non va bene non entra nel progetto, ma resta nella storia e si può correggere e riproporre.

6. Sicurezza e regole di laboratorio

1. Password robuste e mai condivise; attenzione a e-mail sospette (phishing).

2.
Regole "quando hai finito": niente giochi, niente YouTube, niente cose che fanno rumore; resti al tuo posto; blocca/esci col `Ctrl + Alt + Canc`.

3.
Se hai finito: ripassa, fai lo schema a mano sul quaderno, aiuta un compagno, prova a spiegare a voce cosa hai capito.

6.1 Sicurezza 1: pericolo, danno, rischio (22/10)

1. Pericolo: una cosa che **può** fare male (il coltello, l'olio bollente, la presa rotta, il pavimento bagnato, il cavo per terra). Domanda magica: «Che cosa qui mi può fare male?».

2. Danno: il male che **succede davvero** (un taglio, una bruciatura, una caviglia storta, gli occhi stanchi dopo ore di schermo).

3. Rischio: quanto è **facile** che il danno succeda e quanto è **grave**. Lo stesso pericolo ha un rischio diverso a seconda di come lo usi: motorino con il casco allacciato (basso) o senza casco, veloce, con il telefono in mano (alto); telefono in carica sul tavolo (basso) o sotto il cuscino tutta la notte (alto).

4. «Per un pelo» (quasi incidente): non ti sei fatto niente, ma il pericolo c'era: lo dici subito, così il prossimo non si fa male.

5. La prossima volta: **prevenzione** (il danno succede meno spesso) e **protezione** (il danno è meno grave). Documento completo: `classe-1/sicurezza-1ora/1inf-sicurezza-1ora.md`.

7. Esercitazioni svolte

1.
Io e la mia famiglia — presentazione di sé e della famiglia con Google Documenti, consegna su Classroom (anche scrivendo nella propria lingua + traduzione).

2.
Ricerca: cos'è GitHub e cos'è un repository — piccola ricerca in rete, scritta in un documento e consegnata su Classroom.

3.
I componenti del PC — per ogni componente spiegare a cosa serve con parole proprie + due domande personali.

4.
Costruisci il tuo PC — montare un PC compatibile a budget con PCPartPicker (Builder) e scrivere la lista con il prezzo totale.

5.
Documento sulla configurazione del PC — spiegare con parole proprie gli standard (form factor, PCIe, socket, processori per marca, memorie) e inserire immagini dal web descrivendo la tipologia; consegna su Classroom.

7.1 Quanto pesa un file e quanto va veloce la rete (20 e 21/10)

1. Bit e byte: il bit vale 0 oppure 1; 8 bit fanno 1 byte, cioè una lettera. Il bit si scrive con la **b** piccola, il byte con la **B** grande.

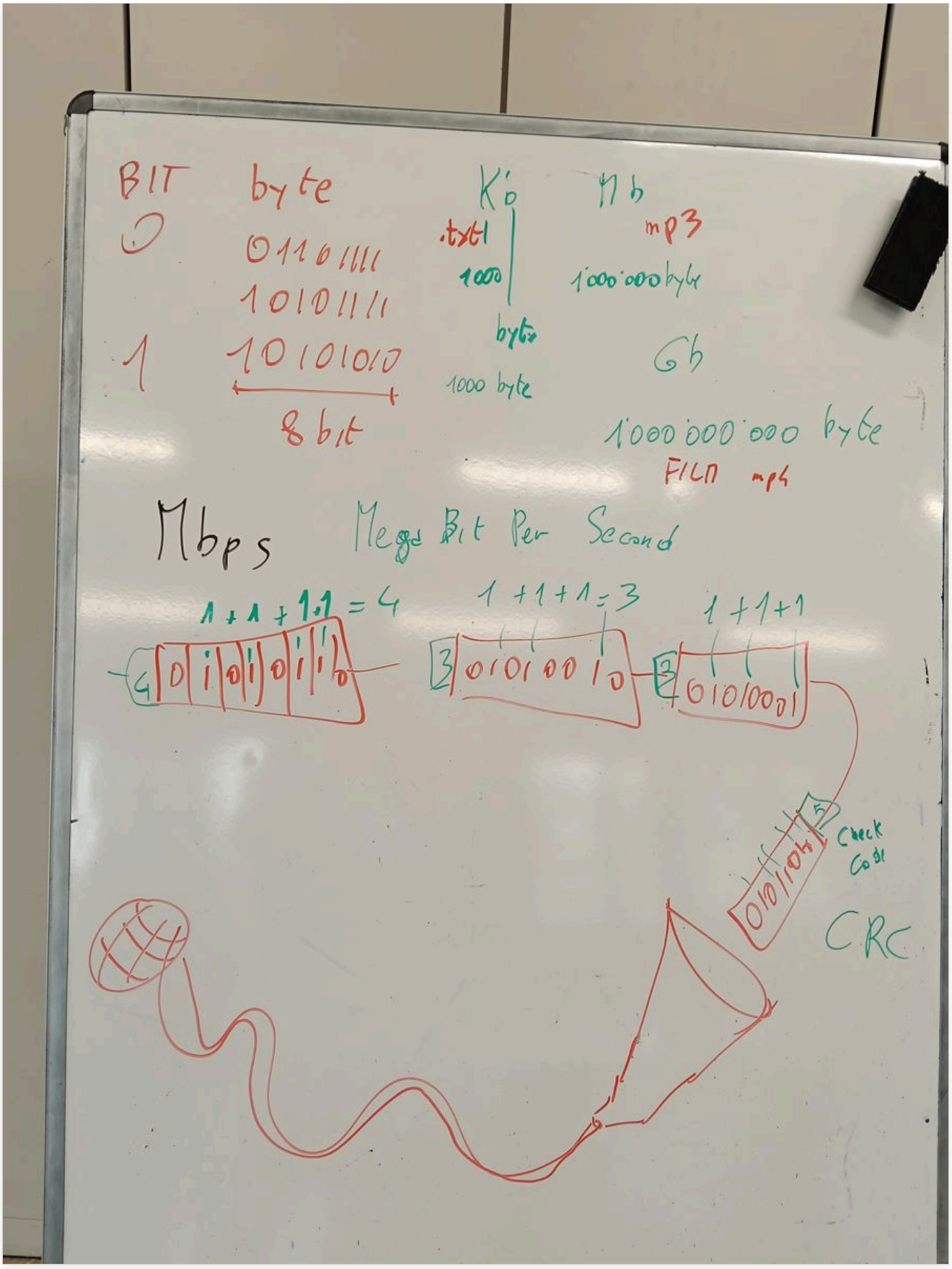
2. Le unità: ogni gradino è circa 1000 volte più grande: KB, MB, GB, TB. Una foto pesa circa 3 MB, un film circa 2 GB.

3. Il conto: prima si porta tutto nella stessa unità, poi si divide. Esempio: 64 GB = 64000 MB; 64000 : 3 = circa 21000 foto.

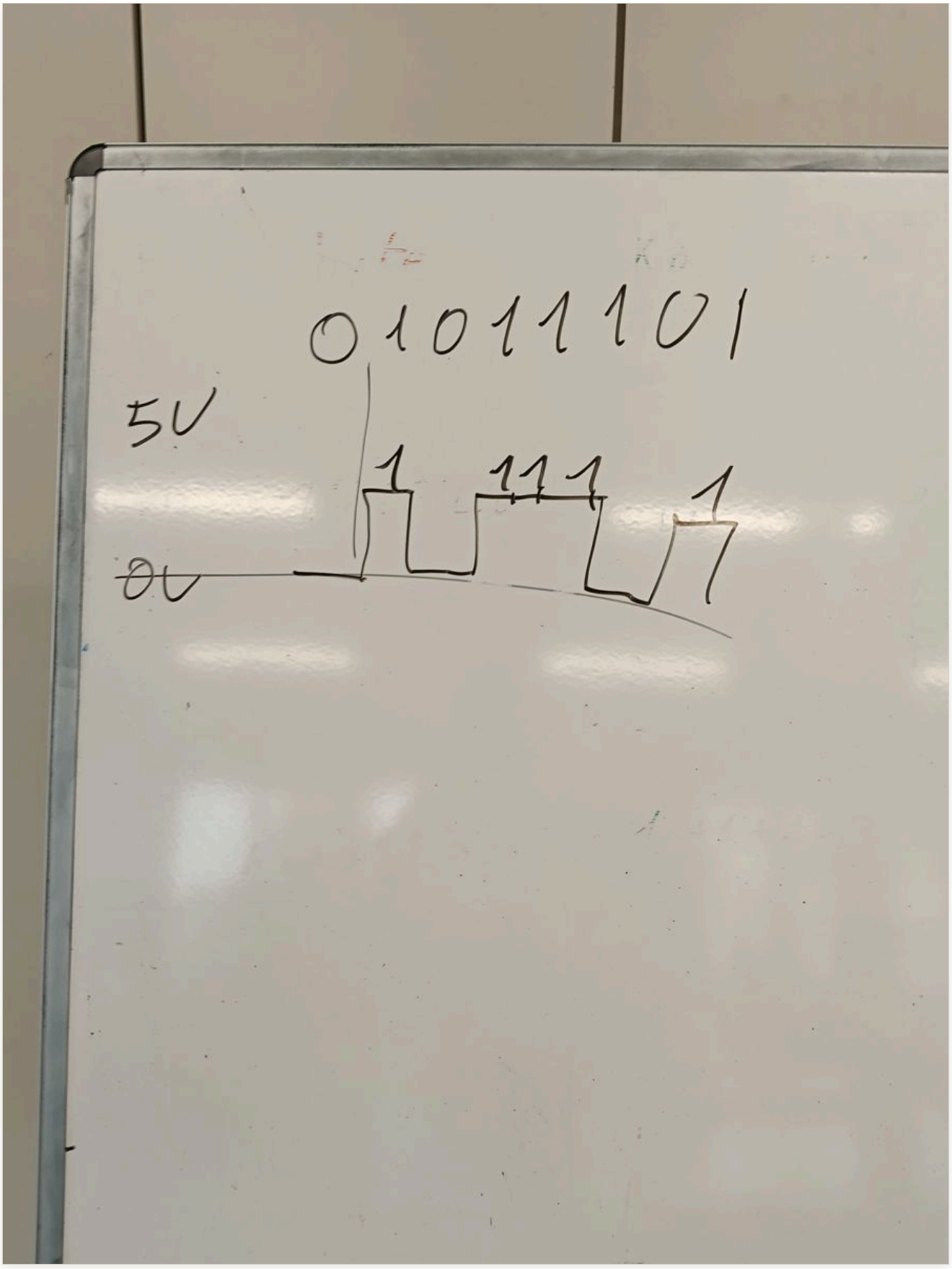
4. La velocità della rete si misura in **Mbps** (megabit al secondo): 100 Mbps = 12,5 MB al secondo.

5. Il controllo dei dati (CRC): chi spedisce aggiunge un numero di controllo, chi riceve rifà il conto; se non torna, un bit si è rovinato e il pacchetto si chiede di nuovo.

6. Nel cavo i bit sono corrente: 5 volt = 1, 0 volt = 0.



La lavagna del 20/10: bit, byte, Mbps e il controllo dei dati



La lavagna del 20/10: il byte diventa corrente nel cavo

1. Il laboratorio del gamer (21/10): il **ping** è il tempo di andata e ritorno verso il server (sotto 30 ms ottimo); il **jitter** è quanto il ping cambia; il **lag** è il ritardo che si sente giocando. Gli **FPS** sono le immagini al secondo prodotte dalla scheda video; gli **Hz** sono quante volte al secondo il monitor ridisegna l'immagine. I cavi video:

VGA (vecchio, analogico), HDMI (porta anche l'audio), DisplayPort (monitor da gaming). Documento completo: `classe-1/gamer/1inf-gamer.md`.

8. Lazarus: il mio primo programma (28/10)

Programmazione · Informatica, laboratorio · 28/10/2026 · la prima finestra con una casella (TEdit) e un bottone (TButton): scrivi il nome, premi e compare «Ciao» con il tuo nome; i file del progetto; consegna su Classroom. Dispensa completa in tre lingue (italiano, arabo, cinese): `classe-1/lazarus-ciao/20261028_Classe-1_v1.0_Lazarus-Primo-Programma_IT.md` (e `_AR`, `_ZH`).

8.1 Che cos'è Lazarus

Lazarus è un programma che serve a **creare altri programmi**, con le finestre e i bottoni, come quelli che usi ogni giorno. Funziona come i **mattoncini**: prendi un pezzo già pronto (un bottone, una casella), lo metti nella finestra e poi scrivi **che cosa deve fare**. Il linguaggio si chiama **Pascal**.

Oggi facciamo il primo programma di tutti i programmatori: il **«Ciao»**. Scrivi il tuo nome in una casella, premi un bottone e il programma ti saluta: **Ciao Ahmed**.

Per aprire Lazarus: doppio clic sull'icona di Lazarus (il prof vi mostra dove si trova). Poi vai al capitolo 02: lì prepari il progetto nuovo e la tua cartella.

8.2 Partenza pulita: nuovo progetto e cartella nuova

Prima di mettere i pezzi nella finestra prepariamo il posto giusto. **Un progetto = una cartella sola**: tutti i file del tuo programma stanno insieme, nella tua cartella. Così nessun file si perde e nessun file si mescola con quelli di un altro.

Fai un passo alla volta. Dopo ogni passo guarda lo schermo.

1. In alto a sinistra in Lazarus clicca il menu «File».

2. Nel menu clicca «Chiudi tutto». Così si chiudono i file rimasti aperti.

3. Se Lazarus chiede di salvare un progetto vecchio, clicca «No».

4. In alto clicca il menu «Progetto».

5. Nel menu clicca «Chiudi progetto». Se la voce è grigia, non si può cliccare: va bene così, vai al passo 6.

6. Ora fai il progetto nuovo. In alto clicca il menu «Progetto».

7. Nel menu clicca «Nuovo progetto…». Si apre una finestra con un elenco.

8. Nell'elenco a sinistra clicca «Applicazione».

9. In basso nella finestra clicca «OK». Al centro compare la finestra vuota **Form1**.

10. SUBITO si salva. In alto clicca il menu «Progetto».

11. Nel menu clicca «Salva progetto con nome…». Si apre la finestra per salvare.

12. In alto nella finestra clicca il bottone «Nuova cartella». (Non lo vedi? Clic con il tasto destro su un punto vuoto della finestra «Nuovo» «Cartella».)

13. Scrivi il nome della cartella e premi `Invio`:

```
ciao
```

14. Fai **doppio clic** sulla cartella `ciao` per entrarci. In alto ora c'è scritto `ciao`.

15. In basso, nella casella «Nome file», scrivi il nome del progetto:

```
ciao.lpi
```

16. In basso a destra clicca «Salva».

17. Lazarus apre una seconda finestra: chiede il nome della **unit** (il file del codice). Nella casella «Nome file» lascia o scrivi:

```
unit1.pas
```

18. In basso a destra clicca «Salva».

19. Controlla: guarda in alto, nella barra del titolo di Lazarus. C'è scritto `ciao`. FATTO: il tuo progetto ha la sua cartella!

8.3 Le finestre di Lazarus

Quando apri un progetto nuovo vedi più finestre. Ognuna ha un lavoro:

Dove	Si chiama	A che cosa serve
in alto	**menu** e **palette dei componenti**	i comandi («File», «Progetto», «Esegui»…) e, sotto, i pezzi da mettere nella finestra, divisi in linguette
a sinistra	**Ispettore oggetti**	le **proprietà** del pezzo selezionato (il testo, il nome, il colore) e i suoi **eventi** (che cosa succede quando lo clicchi)
al centro	**Form1**, con i puntini	la **finestra del tuo programma**: qui metti i pezzi
dietro la Form	**Editor del codice** (unit1.pas)	qui scrivi **che cosa fa** il programma
in basso	**Messaggi**	Lazarus ti dice se è andato tutto bene o dove c'è un errore

Per passare dalla Form al codice e ritorno: tasto `F12`.

Da ricopiare sul quaderno

```
+-------------------------------------------------+
| MENU:  File  Progetto  Esegui ...               |
| PALETTE: [Standard]  [OK] [abc] [A] ...         |
+------------+------------------------------------+
| ISPETTORE  |                                    |
| OGGETTI    |   FORM1  (la mia finestra)         |
|            |    +-----------------+             |
| Proprieta  |    | Edit1           |  <- TEdit   |
|  Caption   |    +-----------------+             |
|  Text      |       [ Saluta ]       <- TButton  |
| Eventi     |                                    |
|  OnClick   |   (F12 = vado al codice)           |
+------------+------------------------------------+
| MESSAGGI: errori e avvisi                       |
+-------------------------------------------------+
```

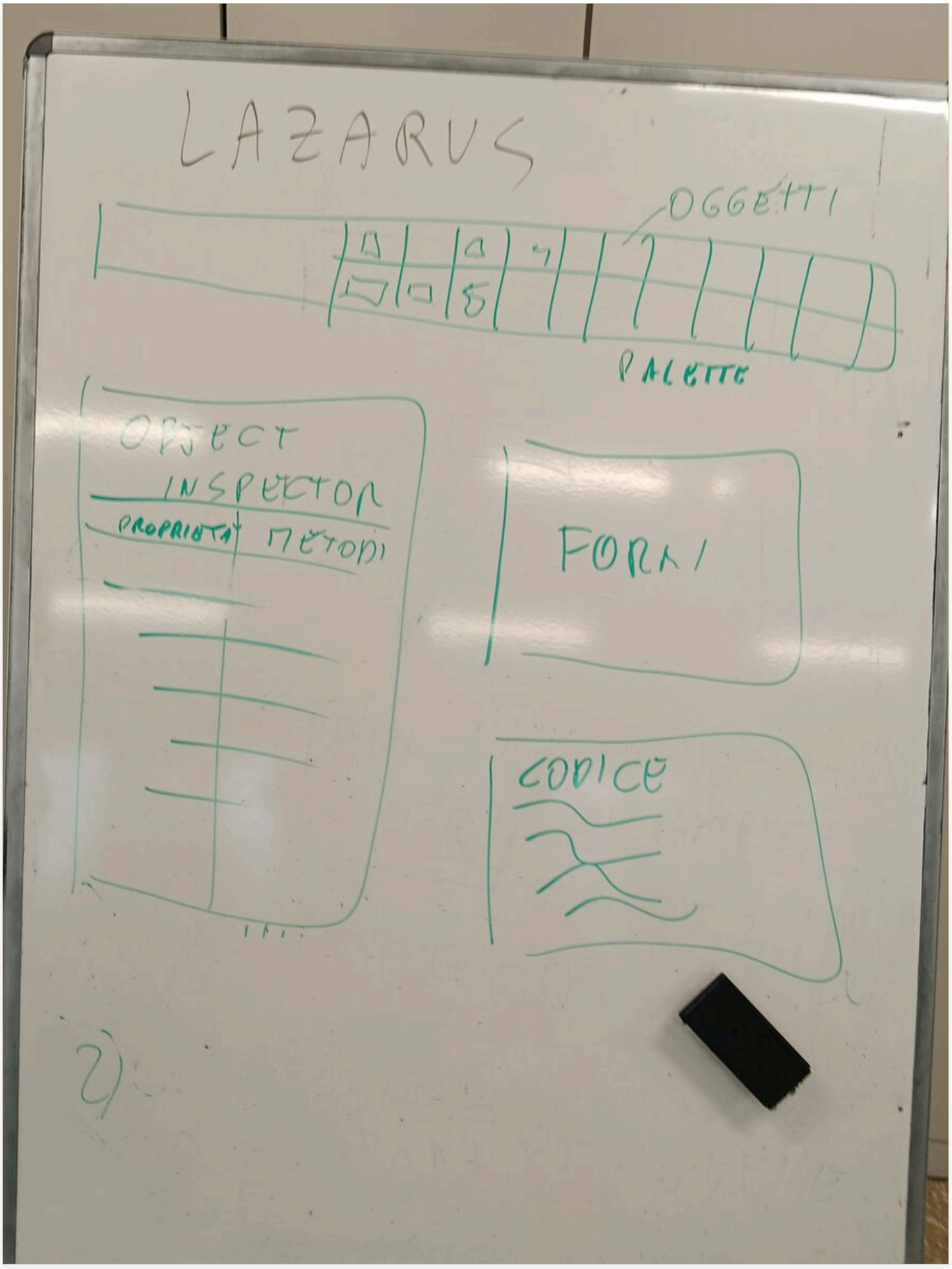


Foto della lavagna: la lavagna del 28/10/2026: le finestre di Lazarus. In alto la palette con gli oggetti (i componenti); a sinistra l'Object Inspector (l'Ispettore oggetti) con le proprietà e i metodi (gli eventi); a destra la Form e, sotto, il codice.

8.4 La palette: come scegliere il pezzo giusto

La **palette dei componenti** è la fila di icone in alto, sotto il menu. I pezzi di oggi sono nella linguetta «Standard». Per scegliere, chiediti: **che cosa deve fare chi usa il programma?**

Deve…	Pezzo (componente)	Icona nella palette	Proprietà che conta
scrivere qualcosa	**TEdit** (casella di testo)	una casella bianca con «abc» o «ab|»	`Text` = quello che c'è scritto dentro
premere per far succedere una cosa	**TButton** (bottone)	un bottone con «OK»	`Caption` = la scritta sul bottone
leggere un testo (solo guardare)	**TLabel** (etichetta)	una «A»	`Caption` = il testo che si vede

Per metterlo nella finestra: **un clic** sull'icona nella palette, poi **un clic** sulla Form1, nel punto dove lo vuoi. Lazarus gli dà un nome da solo: `Edit1`, `Button1`. Il **nome** (`Name`) è quello che userai nel codice.

8.5 Esercitazione: il mio «Ciao»

Un passo alla volta. Dopo ogni passo guarda lo schermo: deve essere uguale.

1. Nella palette, linguetta «Standard», clicca **TEdit**, poi clicca nella Form1, in alto a sinistra. Compare una casella con scritto `Edit1`.

2. Con la casella selezionata, nell'Ispettore oggetti trova la proprietà `Text` e cancella la scritta «Edit1»: la casella resta vuota.

3. Nella palette clicca **TButton**, poi clicca nella Form1 sotto la casella. Compare `Button1`.

4. Con il bottone selezionato, nell'Ispettore oggetti cambia `Caption` in `Saluta`. FATTO: la finestra è pronta!

5. Fai **doppio clic sul bottone**. Si apre il codice, con il cursore qui:

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  // qui scrivi la tua riga
end;
```

6. Tra `begin` e `end;` scrivi questa riga:

```
ShowMessage('Ciao ' + Edit1.Text);
```

Che cosa vuol dire, pezzo per pezzo:

Pezzo	Vuol dire
`ShowMessage( … )`	apre una **finestrella** con un messaggio e il bottone **OK**. Quello che c'è tra le parentesi è il messaggio
`Edit1.Text`	il testo scritto dentro la casella Edit1 (il nome)
`'Ciao '`	una scritta fissa. Va tra apici. Lo **spazio** dopo Ciao serve per non avere «CiaoAhmed»
`+`	attacca due scritte una dopo l'altra
`;`	fine dell'istruzione (come il punto alla fine di una frase)

Tutto insieme: **«apri una finestrella con scritto Ciao e il nome che c'è nella casella»**. Come alla lavagna: scrivi MARIO, premi Saluta, compare **Ciao MARIO** con il bottone OK. È la funzione che risponde al **clic** del bottone: si chiama `Button1Click`.

Da ricopiare sul quaderno

```
scrivo MARIO -> clic su [Saluta] -> OnClick -> Button1Click
   -> ShowMessage('Ciao ' + Edit1.Text) -> | Ciao MARIO  [OK] |
```

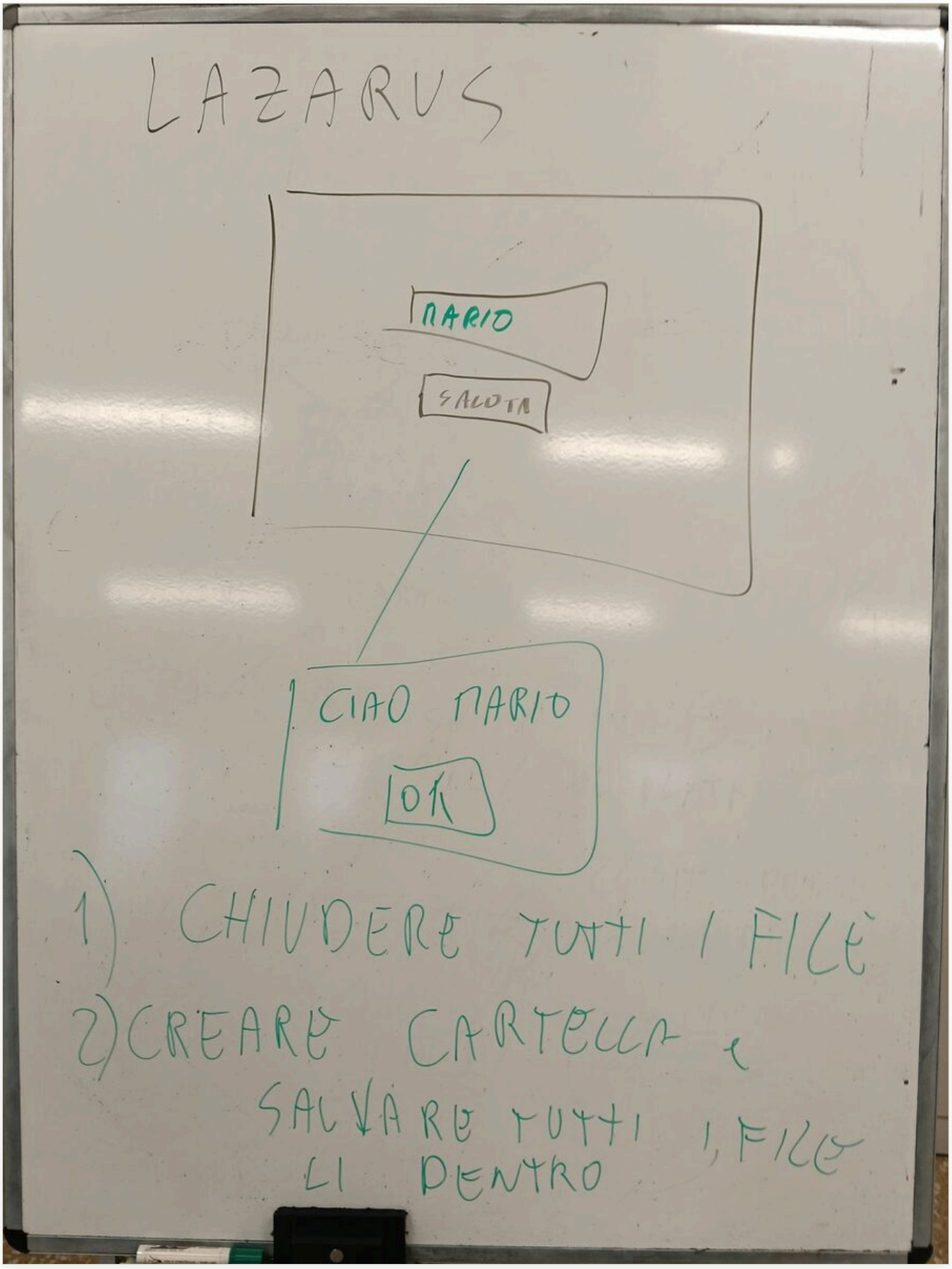


Foto della lavagna: la lavagna del 28/10/2026. In alto la finestra con la casella (MARIO) e il bottone Saluta; il clic apre la finestrella «Ciao MARIO» con il bottone OK. Sotto: 1) chiudere tutti i file; 2) creare la cartella e salvare tutti i file lì dentro.

8.6 Salva e fai partire

La cartella `ciao` l'hai già fatta nel capitolo 02. Adesso basta poco.

1. Premi `Ctrl` + `S`: Lazarus salva tutto nella tua cartella.
2. Premi `F9` (oppure la freccia verde ▶ in alto). Lazarus prepara il programma e lo fa partire: si apre la TUA finestra.
3. Clicca nella casella e scrivi il tuo nome.
4. Clicca il bottone **Saluta**. Si apre una finestrella con **Ciao** e il tuo nome.
5. Clicca **OK**. Ce l'hai fatta: hai scritto il tuo primo programma!

> **NOTA**
>
> **Errore? È normale, succede a tutti i programmatori.** Guarda la finestra «Messaggi» in basso: dice la riga. I più comuni: manca il `;`; hai scritto `=` invece di `:=`; manca un apice `'`; la riga è fuori da begin … end.

8.7 I file del progetto, uno per uno

Dopo `F9` apri la cartella `ciao` con Esplora file. Lazarus ha creato tanti file. L'**estensione** sono le lettere dopo il punto nel nome del file: dice **che tipo di file è**. Non vedi le estensioni? In Esplora file, in alto: «Visualizza» «Mostra» «Estensioni nomi file».

Gruppo 1. Il progetto

File	Che cos'è	Che cosa c'è dentro	Si può cancellare?
`ciao.lpi`	informazioni del progetto (**L**azarus **P**roject **I**nformation) *Come: la scheda del progetto, con l'elenco dei pezzi e le istruzioni per montarli*	un testo in formato **XML** (testo con le etichette tra < e >): quali file fanno parte del progetto e le **opzioni** del progetto e del compilatore, quelle che si cambiano dal menu «Progetto» «Opzioni progetto…». Con doppio clic su questo file riapri il progetto	**NO**
`ciao.lpr`	programma principale (**L**azarus **PR**ogram) *Come: il fischio di inizio della partita: fa partire tutto*	le poche righe in Pascal che fanno partire tutto e aprono Form1. Non si tocca	**NO**
`ciao.lps`	sessione (**L**azarus **P**roject **S**ession) *Come: un segnalibro: la memoria di dove eri rimasto*	quali file erano aperti nell'editor, in che ordine erano le linguette, dov'era il cursore. **Non contiene il programma**: se la cancelli, il tuo lavoro c'è ancora tutto	sì: Lazarus la rifà
`ciao.res`	risorse (**RES**ource) *Come: lo zaino con le cose da portare dentro l'exe: icona e versione*	l'icona e la versione del programma, pronte da mettere dentro l'exe	sì: Lazarus la rifà
`ciao.ico`	icona (**ICO**n) *Come: la foto profilo del programma*	il disegnino del programma, quello che vedi sulla barra di Windows	sì: Lazarus la rifà

Gruppo 2. La finestra e il codice

File	Che cos'è	Che cosa c'è dentro	Si può cancellare?
`unit1.lfm`	la form (**L**azarus **F**or**M**) *Come: la piantina di una stanza: dove stanno i mobili*	**com'è fatta la finestra**: quali pezzi ci sono, dove stanno, quanto sono grandi, Caption e Text	**NO**
`unit1.pas`	la unit (un file di codice in **Pas**cal) *Come: la ricetta: le istruzioni che hai scritto tu*	il **TUO codice**: la procedura `Button1Click` con la riga del Ciao. Una unit può avere anche l'estensione `.pp`	**NO**

Gruppo 3. Il programma finito

File	Che cos'è	Che cosa c'è dentro	Si può cancellare?
`ciao.exe`	eseguibile (**EXE**cutable), nella cartella del progetto *Come: la torta già cotta: si mangia anche senza la ricetta*	il **programma finito**, creato con F9. Si apre anche senza Lazarus	sì: si rifà con F9

Gruppo 4. File di lavoro del compilatore (il **compilatore** è il programma che trasforma il Pascal in un programma che il computer sa eseguire)

File	Che cos'è	Che cosa c'è dentro	Si può cancellare?
`lib\x86_64-win64\`	cartella *Come: il tavolo della cucina dove si preparano i pezzi*	i pezzi intermedi che servono per fare l'exe (qui sotto)	sì: Lazarus la rifà
`.o`	file oggetto (**O**bject) *Come: un ingrediente già cotto, uno solo*	il codice macchina di **una sola** unit, per esempio `unit1.o`	sì
`.ppu`	unit compilata di Free Pascal (**P**ascal **P**recompiled **U**nit) *Come: un pezzo già montato, con l'etichetta che dice che cosa sa fare*	le informazioni sulla unit già tradotta, per esempio `unit1.ppu`	sì
`.compiled`	promemoria della compilazione *Come: lo scontrino: ricorda com'è andata l'ultima volta*	come è stato compilato il progetto l'ultima volta: così Lazarus sa se deve rifare tutto	sì
`.or`	risorse compilate *Come: lo zaino del .res già chiuso, pronto da caricare*	il file `.res` già pronto da attaccare all'exe	sì
`backup\`	cartella delle copie di sicurezza *Come: la fotocopia del compito prima di correggerlo*	quando salvi un file che esisteva già, Lazarus mette qui la versione di prima. Le copie dei file del progetto hanno **lo stesso nome** dell'originale (`unit1.pas`, `unit1.lfm`, `ciao.lpi`…); un file che non fa parte del progetto prende in più `.bak` (per esempio `prova.pas.bak`). La cartella compare dal secondo salvataggio	sì

Con le impostazioni normali di Lazarus 4, fuori dalla cartella `backup` non trovi copie con `~` o `.bak`. Se le vedi, qualcuno ha cambiato le impostazioni delle copie (menu «Strumenti» «Opzioni…», pagina «Backup»).

Regola da ricordare: i **4 file da tenere**, da consegnare e da mettere su GitHub sono `.lpi` `.lpr` `.pas` `.lfm`. Tutto il resto Lazarus lo rifà da solo.

Da ricopiare sul quaderno

```
ciao\                       <- UNA cartella = UN progetto
 |- ciao.lpi    progetto (+ opzioni)      TENERE
 |- ciao.lpr    programma principale      TENERE
 |- unit1.pas   il mio codice             TENERE
 |- unit1.lfm   com'e' fatta la finestra  TENERE
 |- ciao.lps    sessione                  si rifa'
 |- ciao.res    risorse (icona)           si rifa'
 |- ciao.ico    icona                     si rifa'
 |- ciao.exe    programma finito (F9)     si rifa'
 |- lib\x86_64-win64\
 |     .o .ppu .compiled .or (compilatore) si rifa'
 |- backup\     copie di sicurezza        si butta
```

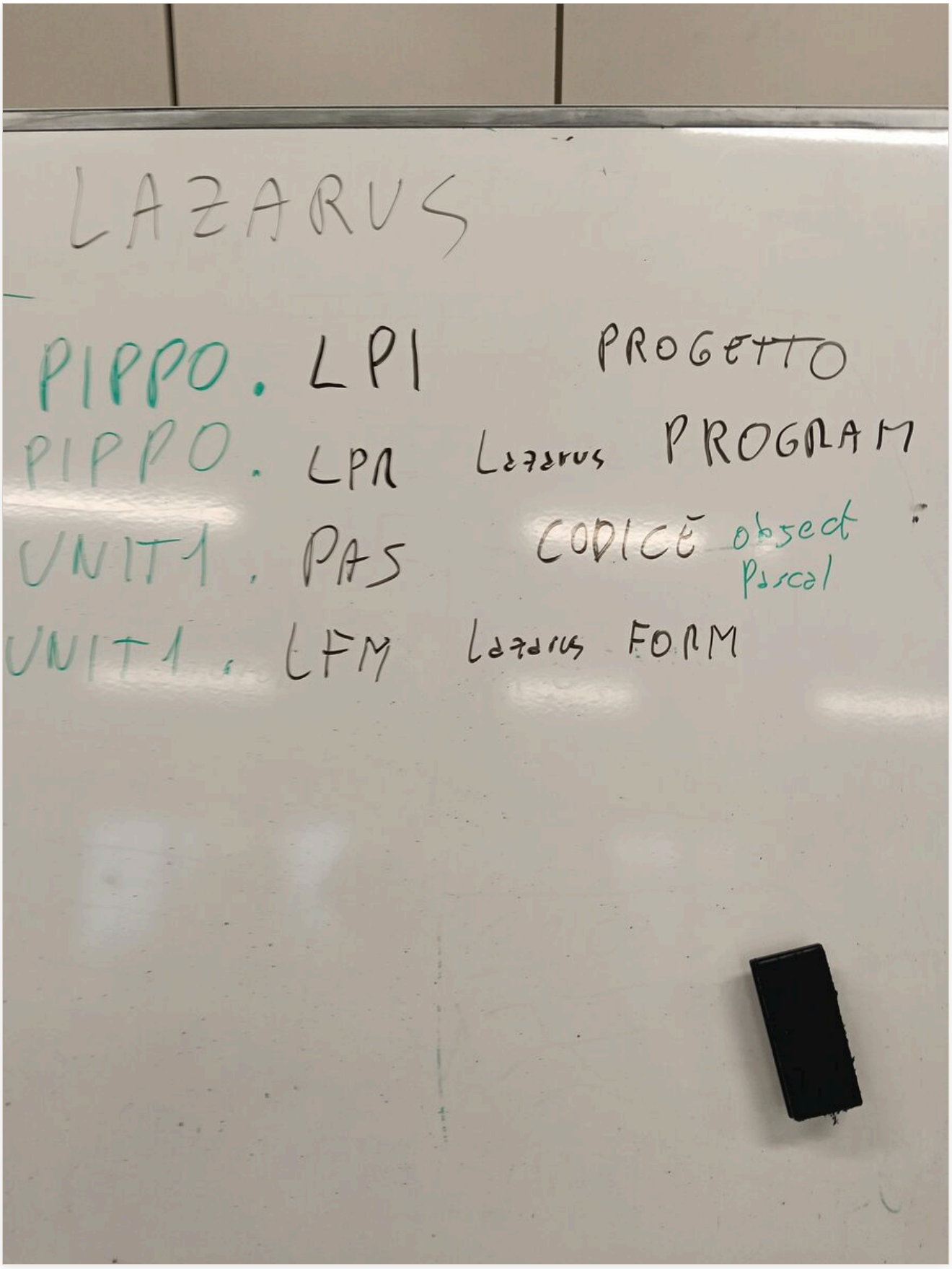


Foto della lavagna: la lavagna del 28/10/2026: i 4 file da tenere, con un progetto che si chiama PIPPO. PIPPO.LPI è il progetto; PIPPO.LPR è il Lazarus Program (il programma principale); UNIT1.PAS è il codice in Object Pascal; UNIT1.LFM è la Lazarus Form (com'è fatta la finestra).

8.8 Fallo tuo

Il programma funziona? Adesso rendilo **tuo**. Scegli almeno una cosa:

1. Il saluto nella **tua lingua**: al posto di `'Ciao '` scrivi `'Hello '`, `'مرحبا '`, `'你好 '`, `'Hola '`…
2. Il **titolo** della finestra: clicca su un punto vuoto della Form1, nell'Ispettore oggetti cambia `Caption` (per esempio «Il saluto di Ahmed»).
3. Il **colore**: sempre sulla Form1, proprietà `Color`, scegli un colore dalla tendina.
4. Per i più veloci: aggiungi una TLabel e una seconda riga, `Label1.Caption := 'Ciao ' + Edit1.Text;`: il saluto resta scritto anche nella finestra.

Poi **spiegalo in 30 secondi** al compagno: «Ho messo una TEdit e un TButton; quando clicco, la funzione Button1Click apre una finestrella con Ciao e il nome».

8.9 Consegna

Su Classroom c'è il compito **«Il mio primo programma: Ciao in Lazarus»**. Dentro c'è un **Documento Google** già pronto, solo tuo. Aprilo e fai 3 cose:

1. **Il codice.** In Lazarus seleziona con il mouse la tua procedura, da `procedure TForm1.Button1Click` fino a `end;`, premi `Ctrl` + `C`; nel Documento clicca sotto «1. Il mio codice» e premi `Ctrl` + `V`.
2. **Lo screenshot.** Fai partire il programma (`F9`), scrivi il tuo nome e premi Saluta. Poi premi `Win` + `Shift` + `S` e trascina un rettangolo intorno alla tua finestra. Nel Documento clicca sotto «2. Il mio programma che funziona» e premi `Ctrl` + `V`.
3. **Con parole tue.** Sotto «3» scrivi in una o due frasi che cosa fa la riga `ShowMessage('Ciao ' + Edit1.Text);`. Puoi scrivere anche nella tua lingua.

Poi in Classroom premi «Consegna» (il bottone blu in alto a destra nel compito).

9. Glossario (essenziale)

1. **CPU:** processore, il "cervello" del PC.
2. **RAM:** memoria di lavoro veloce e temporanea.
3. **SSD/Hard Disk:** memoria permanente dove si salvano i file.
4. **Repository:** cartella-progetto con i file e la storia delle modifiche.
5. **Commit:** un salvataggio (una "fotografia") delle modifiche fatte.
6. **Compatibilità:** quando due pezzi funzionano bene insieme.
7. **Lazarus:** il programma con cui si creano altri programmi con finestre e bottoni; il linguaggio è il Pascal.
8. **Compilatore:** il programma che trasforma il codice Pascal in un programma che il computer sa eseguire (il file .exe).
9. **Estensione:** le lettere dopo il punto nel nome di un file (.pas, .lpi, .exe): dicono che tipo di file è. (Il glossario completo multilingue è nel documento `glossario-l2`.)

10. Changelog

1. **v0.9 (28/10/2026)**: aggiunto il capitolo 8 "Lazarus: il mio primo programma" (Lazarus e le sue finestre, la palette, l'esercitazione «Ciao» con ShowMessage, salva e fai partire, i file del progetto spiegati uno per uno, fallo tuo, consegna su Classroom; tre foto della lavagna del 28/10 e gli schemi da ricopiare sul quaderno); glossario e changelog diventano i capitoli 9 e 10; nel glossario Lazarus, compilatore, estensione.
2. **v0.8 (22/10/2026)**: aggiunta 6.1 "Sicurezza 1: pericolo, danno, rischio" (prima lezione del corso sulla sicurezza, con casi della vita di tutti i giorni).
3. **v0.7 (21/10/2026)**: aggiunta 7.1 "Quanto pesa un file e quanto va veloce la rete" (bit, byte, unità, Mbps, CRC, segnale 0V/5V, laboratorio del gamer) con le due foto della lavagna del 20/10.
4. **v0.6 (09/10/2026)**: aggiunta 5.2 "La lavagna del 09/10" (numeri major.minor.patch.build, ramo principale, changelog, fork, Pull Request accettata e rifiutata) con la foto.

5. v0.5 (09/10/2026): aggiunta 5.1 “La storia del nostro gioco: versioni, rami, fork e merge” (grafo del Treno blindato, versioni minor e major, Godot v2.0).

6. v0.4 (24/09/2026): aggiunta 2.4.1 “In parole semplici — la storia dello storage” (paginetta descrittiva con analogia).

7. v0.3 (24/09/2026): aggiunta 2.4 “Lo storage: i 5 livelli” (HDD/SSD, SATA/NVMe, AHCI/NVMe, connettori SATA/M.2, interfaccia PCIe) con infografica.

8.

v0.2 (24/09/2026): aggiunta 2.3 “Standard e compatibilità” (form factor, PCIe, socket, processori per marca, RAM) e l’esercitazione “Documento sulla configurazione del PC”.

9.

v0.1 (21/09/2026): prima versione. Come lavoriamo; cos’è l’informatica; il computer e le sue parti (+ assemblaggio + scelta pezzi); utenze e aree; Google Suite e Classroom; GitHub e repository; sicurezza e regole di laboratorio; elenco esercitazioni svolte; glossario essenziale.